

프로젝트 보고서 - 기말

날짜: 2026-06-17, 학번: 214466, 이름: 김건희

1. 프로젝트 개요

프로젝트 이름

SpamCheck - AI 기반 스팸 판별 웹 서비스 (MLOps 파이프라인 통합)

프로젝트 목적

FastAPI 기반의 스팸 판별 웹 서비스를 구축하고, 실제 ML 모델(TF-IDF + Naive Bayes)을 학습·등록·배포하는 **완전한 MLOps 파이프라인**을 구현하는 것이 목적이다.

- 사용자가 텍스트를 입력하면 스팸 여부와 확률을 실시간으로 반환
- GitHub Actions로 코드 변경 시 자동 테스트 · Docker 빌드 검증 · 모델 재학습
- MLflow Model Registry로 모델 버전 관리, 승격(Staging→Production), 롤백 지원
- Docker로 패키징하여 Render.com에 배포

GitHub 주소 (public 공개 필수)

https://github.com/logan0741/gunhee_homework

> Public으로 변경 필요: GitHub → Settings → Danger Zone → Change repository visibility → Public

배포 주소 및 캡처

배포 주소: <https://gunhee-homework.onrender.com>

MLflow Tracking Server 주소 및 캡처

로컬 MLflow 서버: <http://127.0.0.1:5001>

실행 방법:

```
cd "2week/1_inital SpamCheck"  
python -m mlflow server --host 127.0.0.1 --port 5001
```

mlflow2.17.2ExperimentsModels

Experiments

Search Experiments

☐ Default

☒ spamcheck-ml

☐ spamcheck-auto-train

spamcheck-ml

[Provide Feedback](#)

[Add Description](#)

Share

RunsEvaluationExperimentalTracesExperimental

metrics.rmse < 1 and params.model = "tree"

Time created

State: Active

Datasets

+ New run

Sort: CreatedColumnsGroup by

	Run Name	Created	Dataset	Duration	Source	Models
☐	skillful-hen-254	15 hours ago	-	8.6s	train.py	SpamClassifier v3
☐	wise-snipe-62	15 hours ago	-	8.9s	train.py	SpamClassifier v2
☐	judicious-sow-73	15 hours ago	-	15.0s	train.py	SpamClassifier v1

3 matching runs

mlflow2.17.2ExperimentsModels

Registered Models

[Create Model](#)

Filter registered models by name or tags

Name

Latest version

SpamClassifier

Version 3

Flexible, governed deployments with the new Model Registry UI

Previous experience

Ver 2Production

Ver 1Staging

New Model Registry UI

Ver 2@Champion@LatestApproved

Ver 1@ChallengerIn-Review

With the latest Model Registry UI, you can use **Model Aliases** for flexible references to specific model versions, streamlining deployment in a given environment. Use **Model Tags** to annotate model versions with metadata, like the status of pre-deployment checks.

Learn more

Try it now

New model registry UI

< Previous

Next >

25 / page

2. 소프트웨어 주요 기능

사용자가 이용할 수 있는 핵심 기능

기능	설명
스팸 판별	텍스트 입력 → 스팸/정상 분류 + 확률값 반환
모델 버전 확인	<code>`/model/info`</code> API로 현재 사용 중인 모델 버전 조회
모델 핫 리로드	<code>`/model/reload`</code> API로 서버 재시작 없이 신규 모델 반영
장애 자동 보고	서버 오류 발생 시 GitHub Issue 자동 생성

Web UI: 텍스트 입력 → 분류 버튼 → 결과 색상 표시 (빨간색: 스팸, 초록색: 정상) + 확률 %

ML 모델이 사용되는 위치

- `app/spam.py` 의 `check_spam()` 함수: 입력 텍스트에 대해 TF-IDF 변환 후 확률 예측
- `POST /classify` 엔드포인트: 사용자 요청 수신 → `check_spam()` 호출 → JSON 응답
- 서버 시작 시 `models/spam_pipeline.pkl` 을 메모리에 로드하고 요청마다 재사용

입력 데이터와 출력 결과

입력:

```
POST /classify
{ "text": "You have won a FREE prize click here now" }
```

출력:

```
{ "label": "spam", "score": 0.9342 }
```

- `label`: "spam" 또는 "ham"
- `score`: 스팸 확률 (0.0 ~ 1.0, ML 모델 사용 시 확률값 / 키워드 폴백 시 키워드 비율)

서비스와 ML 기능 분리

레이어	역할	파일
서비스 레이어	HTTP 요청·응답, 로깅, GitHub 이슈	`app/main.py`
ML 레이어	모델 로드, 예측 수행, 버전 관리	`app/spam.py`
학습 레이어	데이터 로드, 모델 학습, MLflow 등록	`train.py`

3. 실행 환경

항목	내용
사용 OS 및 버전	Windows 11 Pro 10.0.26200
Python (로컬)	Python 3.8.10
Python (Docker)	Python 3.10-slim
Git/GitHub	Git 2.x, GitHub 저장소 `logan0741/gunhee_homework`
Docker	Docker 29.2.1
MLflow	MLflow 2.17.2 (scikit-learn 1.3.2)
배포 환경	Render.com (Docker 기반)
주요 패키지	fastapi 0.135.1, uvicorn 0.41.0, scikit-learn ≥1.3.0, mlflow ≥2.17.0

4. 전체 MLOps 파이프라인 구조

개발자 코드 수정

|



git push (master/main)

|

└─> [CI Workflow - ci.yml]

|

pytest 9개 테스트 자동 실행

|

실패 시 머지 차단

|

└─> [Docker Verify - docker-verify.yml]

|

Docker 이미지 빌드 검증

|

컨테이너 실행 + 헬스체크

```
|
└─> [MLflow Auto Train - mlflow-train.yml]
    TF-IDF + Naive Bayes 모델 학습
    MLflow에 파라미터/메트릭/아티팩트 기록
    MLflow Model Registry에 SpamClassifier 등록
    models/spam_pipeline.pkl 커밋 → 저장소 반영
    |
    ▼
    Render.com 자동 배포
    (master push 시 Docker 이미지 재빌드)
    |
    ▼
    /model/reload API로 새 모델 핫 로드
```

코드 변경 흐름

로컬 수정 → `git push` → CI 자동 실행 → 테스트 통과 → Docker 검증 → Render 재배포

모델 학습 흐름

`git push` → `mlflow-train.yml` 트리거 → MLflow 서버 시작 → `train.py` 실행 → 모델 학습 → MLflow 등록 → `models/spam_pipeline.pkl` 저장 → git commit → push

모델 등록/반영 흐름

학습 완료 → MLflow Registry에 `SpamClassifier` 신규 버전 생성 → `scripts/promote.py` 로 Production 승격 → 서비스에서 `/model/reload` 호출 → 신규 모델 적용

서비스 운영 흐름

사용자 요청 → FastAPI 수신 → 로그 기록 → ML 모델 예측 → 응답 반환 → 오류 시 GitHub Issue 자동 생성

5. Git 기반 개발 과정

개발 흐름

초기 FastAPI 기본 구조를 `master` 브랜치에 작성한 후, 기능별 커밋 단위로 점진적으로 추가하였다.

- 1. FastAPI 초기 설정 및 키워드 기반 spam.py 구현
- 2. Docker 패키징 (Dockerfile, .dockerignore, compose.yaml)
- 3. GitHub Actions 3종 추가 (ci.yml, docker-verify.yml, mlflow-train.yml)
- 4. 로깅 및 GitHub Issue 자동 생성 (logging, issue.py)
- 5. MLflow 실험 추적 및 Model Registry 연동
- 6. [기말] TF-IDF + Naive Bayes ML 모델 업그레이드
- 7. [기말] 자동 승격 워크플로우, 롤백 스크립트 추가

커밋 전략

접두사	사용 시점	예시
<code>`feat:`</code>	새 기능 추가	<code>`feat: add /classify endpoint`</code>

`fix:`	버그 수정	`fix: Python 3.8 compatibility`
`chore:`	설정, 빌드, 자동화	`chore: update trained model`
`docs:`	문서 작성	`docs: add report`

브랜치 사용

- 기본 브랜치: `master` / 제출 브랜치: `main`
- 주요 기능은 `master` 직접 커밋 (소규모 프로젝트)
- GitHub Actions 워크플로우 검증 후 `master` 반영

주요 커밋 이력:

```
10fad47 fix: Python 3.8 compatibility
e58c084 fix: MLFlow server inside Actions runner
edfc688 Add MLFlow auto train workflow
c2c4e4d Add logging and GitHub issue reporting
13047e4 Dockerize for deployment
```

6. CI/CD 구성

GitHub Actions 구성

총 4개의 워크플로우로 CI/CD 파이프라인을 구성하였다.

워크플로우	파일	트리거
CI (테스트)	`ci.yml`	push/PR to master-main
Docker Verify	`docker-verify.yml`	push/PR to master-main
MLflow Auto Train	`mlflow-train.yml`	push to master-main, 수동
Auto Promote	`auto-promote.yml`	수동 (workflow_dispatch)

테스트/빌드/배포 자동화

- **테스트 자동화:** `pytest` 9개 테스트, push마다 자동 실행
- **빌드 자동화:** Docker 이미지 빌드 및 컨테이너 헬스체크 자동 검증
- **배포 자동화:** Render.com의 GitHub 연동으로 `master` push 시 자동 재배포
- **모델 학습 자동화:** push 시 TF-IDF + NB 모델 학습 후 git 커밋

workflow 주요 단계 설명

ci.yml

```
steps:
  - Checkout
  - Python 3.10 설정
  - pip install -r requirements.txt
  - pytest # 9개 테스트 자동 실행
```

mlflow-train.yml

```
steps:
  - Checkout
  - Python 설정 + 의존성 설치
  - MLflow 서버 시작 (백그라운드)
  - train.py 실행 (TF-IDF+NB 학습, MLflow 등록)
  - 학습된 모델 git commit & push
```

auto-promote.yml (수동)

```
inputs:
  accuracy_threshold: "0.90" # 이 값 이상이면 Production 승격
steps:
  - 학습 실행
  - scripts/promote.py 실행 → 정확도 기준 자동 승격
```

실행 결과 캡처

Platform Solutions Resources Open Source Enterprise Pricing

Search or jump to... / Sign in Sign up

logan0741 / gunhee_homework Public

Notifications Fork 0 Star 0

<> Code Issues 1 Pull requests Actions Projects Security and quality Insights

Actions

All workflows

Auto Promote Model

CI

Dependency Graph

Docker Verify

MLFlow Auto Train

Management

Caches

All workflows

Showing runs from all workflows

22 workflow runs

Workflow Event Status Branch Actor

Graph Update: pip in /. #1420148949

Dependency Graph #1: by dependabot Bot

main

6 minutes ago

36s

feat: upgrade to ML model (TF-IDF + Naive Bayes) with full ML...

Docker Verify #7: Commit 4aa787d pushed by logan0741

main

Today at 4:05 PM

1m 24s

...

feat: upgrade to ML model (TF-IDF + Naive Bayes) with full ML...

MLFlow Auto Train #7: Commit 4aa787d pushed by logan0741

main

Today at 4:05 PM

1m 18s

...

feat: upgrade to ML model (TF-IDF + Naive Bayes) with full ML...

CI #7: Commit 4aa787d pushed by logan0741

main

Today at 4:05 PM

57s

...

feat: upgrade to ML model (TF-IDF + Naive Bayes) with full ML...

CI #6: Commit 4aa787d pushed by logan0741

master

Today at 4:05 PM

1m 5s

...

feat: upgrade to ML model (TF-IDF + Naive Bayes) with full ML...

MLFlow Auto Train #6: Commit 4aa787d pushed by logan0741

master

Today at 4:05 PM

1m 28s

...

feat: upgrade to ML model (TF-IDF + Naive Bayes) with full ML...

Docker Verify #6: Commit 4aa787d pushed by logan0741

master

Today at 4:05 PM

1m 18s

...

mlflow 2.17.2 Experiments Models

GitHub Docs

spamcheck-ml >

skillful-hen-254

Model registered

Overview Model metrics System metrics Artifacts

Description

No description

Details

Created at	2026-06-17 15:45:40
Created by	logan
Experiment ID	681121772289120380
Status	Finished
Run ID	25517d2683fa4155ba169ba7b2547d6b
Duration	8.6s
Datasets used	—
Tags	Add
Source	train.py 10fad47
Logged models	sklearn
Registered models	SpamClassifier v3

Parameters (6)

Parameter	Value
max_features	1000
model_type	TF-IDF + MultinomialNB

Metrics (8)

Metric	Value
v1_accuracy	0.85
v1_f1	0.8235294117647058

7. Docker 기반 환경 구성

Dockerfile 설명

```
FROM python:3.10-slim          # 경량 Python 3.10 이미지

ENV PYTHONDONTWRITEBYTECODE=1  # .pyc 파일 생성 방지
ENV PYTHONUNBUFFERED=1         # 로그 즉시 출력
ENV PORT=10000                 # Render.com 기본 포트

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt # 레이어 캐시 활용

COPY . .

EXPOSE 10000
CMD ["sh", "-c", "uvicorn app.main:app --host 0.0.0.0 --port ${PORT}"]
```

주요 설정 이유:

- `python:3.10-slim`: 최소 크기의 안정적 Python 환경 (개발 도구 제외)
- `requirements.txt` 먼저 복사: 코드만 변경 시 pip install 레이어 재사용
- `PORT` 환경변수: Render.com이 포트를 동적으로 지정하는 방식 지원

MLflow 서버 Dockerfile (`mlflow-server/Dockerfile`)

```
FROM python:3.10-slim
RUN pip install mlflow==2.22.0
WORKDIR /mlflow
EXPOSE 5000
CMD ["mlflow", "server",
    "--host", "0.0.0.0", "--port", "5000",
    "--backend-store-uri", "sqlite:///mlflow/data/mlflow.db",
    "--default-artifact-root", "/mlflow/artifacts"]
```

Render.com에 별도 서비스로 배포하면 공개 MLflow 추적 서버 URL 확보 가능

실행 방법

```
# 로컬 단일 서비스 실행
docker run --rm -p 10000:10000 spamcheck:local

# Docker Compose (FastAPI + MLflow 서버 함께 실행)
docker compose up

# 접속
http://localhost:10000/      # 웹 UI
http://localhost:10000/docs  # Swagger UI
http://localhost:5000/      # MLflow UI
```

사용 데이터

직접 구성한 레이블 데이터셋 (`data/dataset.py`):

- 총 100개: 스팸(1) 50개 + 정상(0) 50개
- 학습/테스트 분할: 80:20 (stratified split, random_state=42)
- 스팸 유형: 복권 당첨, 클릭 유도, 무료 상품, 피싱, 도박, 다이어트 광고 등
- 정상 유형: 일상 메시지, 업무 일정, 가족 연락, 캘린더 알림 등

모델 종류

`scikit-learn` 의 **Pipeline** (TfidfVectorizer + MultinomialNB)

```
Pipeline([
    ('tfidf', TfidfVectorizer(
        max_features=1000,      # 상위 1000개 단어
        ngram_range=(1, 2),    # 단어 단위 + 2-gram
        stop_words='english',   # 영어 불용어 제거
    )),
    ('clf', MultinomialNB(alpha=1.0)), # 라플라스 스무딩
])
```

학습 코드 설명

`train.py` 의 주요 흐름:

1. `data/dataset.py` 에서 100개 레이블 데이터 로드
2. `train_test_split` 으로 80:20 분할
3. TF-IDF 벡터화 후 MultinomialNB 학습
4. 테스트셋 평가 (accuracy, precision, recall, f1)
5. V1(키워드 방식) 대비 성능 비교 MLflow에 기록
6. `mlflow.sklearn.log_model` 로 MLflow Model Registry 등록
7. `models/spam_pipeline.pkl` 로 로컬 저장

평가 지표

지표	설명
Accuracy	전체 정확도
Precision	스팸으로 예측한 것 중 실제 스팸 비율
Recall	실제 스팸 중 스팸으로 예측한 비율
F1-Score	Precision과 Recall의 조화평균

초기 모델(V1)과 신규 모델(V2) 비교

구분	V1: 키워드 방식	V2: TF-IDF + Naive Bayes
방법	스팸 키워드 14개 카운팅 (2개 이상 = spam)	TF-IDF 벡터화 + 확률 기반 분류
파일	<code>`app/spam.py`</code> (<code>check_spam_keyword</code>)	<code>`models/spam_pipeline.pkl`</code>

Accuracy	**85.0%**	**100.0%**
F1-Score	**82.4%**	**100.0%**
출력	키워드 개수 (int)	스팸 확률 (float, 0~1)
신규 패턴 대응	어려움 (키워드 추가 필요)	문맥 학습으로 자동 대응

8. MLflow 기반 실험 관리

MLflow Tracking 사용 여부

사용함. 실험 이름: spamcheck-ml

기록한 항목

Parameters (파라미터):

파라미터	값 (실험 1 기준)
model_type	TF-IDF + MultinomialNB
max_features	1000
ngram_range	(1,2)
nb_alpha	1.0

train_samples	80
test_samples	20

Metrics (메트릭):

메트릭	실험1	실험2	실험3(최종)
v2_accuracy	1.000	1.000	1.000
v2_precision	1.000	1.000	1.000
v2_recall	1.000	1.000	1.000
v2_f1	1.000	1.000	1.000
v1_accuracy	0.850	0.850	0.850
v1_f1	0.824	0.824	0.824

Artifacts (아티팩트):

- spam_pipeline/ (MLflow sklearn 모델 포맷)
- models/spam_pipeline.pkl (pickle 파일)

Model (모델):

- 등록명: SpamClassifier
- 버전: 1, 2, 3

실험 결과 및 화면 캡처

mlflow2.17.2

ExperimentsModels

Experiments

Search Experiments

Default

spamcheck-ml

spamcheck-auto-train

spamcheck-ml

Provide Feedback

Add Description

Share

Runs

Evaluation

Experimental

Traces

Experimental

metrics.rmse < 1 and params.model = "tree"

Time created

State: Active

Datasets

+ New run

Sort: Created

Columns

Group by

	Run Name	Created	Dataset	Duration	Source	Models
	skillful-hen-254	15 hours ago	-	8.6s	train.py	SpamClassifier v3
	wise-snipe-62	15 hours ago	-	8.9s	train.py	SpamClassifier v2
	judicious-sow-73	15 hours ago	-	15.0s	train.py	SpamClassifier v1

3 matching runs

mlflow2.17.2

ExperimentsModels

spamcheck-ml

skillful-hen-254

Model registered

Overview

Model metrics

System metrics

Artifacts

Description

No description

Details

Created at	2026-06-17 15:45:40
Created by	logan
Experiment ID	681121772289120380
Status	Finished
Run ID	25517d2683fa4155ba169ba7b2547d6b
Duration	8.6s
Datasets used	—
Tags	Add
Source	train.py10fad47
Logged models	sklearn
Registered models	SpamClassifier v3

Parameters (6)

Search parameters

Parameter	Value
max_features	1000
model_type	TF-IDF + MultinomialNB

Metrics (8)

Search metrics

Metric	Value
v1_accuracy	0.85
v1_f1	0.8235294117647058

mlflow 2.17.2

Experiments

Models

Registered Models >

SpamClassifier

Created Time: 2026-06-17 15:40:32

Last Modified: 2026-06-17 15:52:31

Description

Edit

Tags

Versions

Compare

Version	Registered at
Version 3	2026-06-17 15:45:48
Version 2	2026-06-17 15:45:19
Version 1	2026-06-17 15:40:32

Flexible, governed deployments with the new Model Registry UI

Previous experience

Ver 2

Production

Ver 1

Staging

New Model Registry UI

Ver 2

@Champion

@Latest

Approved

Ver 1

@Challenger

In-Review

With the latest Model Registry UI, you can use **Model Aliases** for flexible references to specific model versions, streamlining deployment in a given environment. Use **Model Tags** to annotate model versions with metadata, like the status of pre-deployment checks.

Learn more

Try it now

New model registry UI

Description

1

가장 좋은 모델 선정 기준

- `v2_accuracy ≥ 0.90` : Staging 승격
- `v2_accuracy ≥ 0.90` (임계값 설정 기준): Production 승격
- `scripts/promote.py` 실행 시 최신 실험 결과를 조회하여 자동 판단
- 현재 Production: SpamClassifier **v3** (accuracy=1.0000)

9. 모델 등록 및 서비스 반영

모델 저장 방식

1. **MLflow Model Registry:** `mlflow.sklearn.log_model(..., registered_model_name="SpamClassifier")`

- 실험마다 새 버전 자동 생성 (v1 → v2 → v3)
- 각 버전에 Stage 부여 가능 (None / Staging / Production / Archived)

2. **로컬 파일:** `models/spam_pipeline.pkl` (pickle 직렬화)

- Git에 커밋하여 배포 환경에서 즉시 로드 가능

서비스가 모델을 불러오는 방식

```
# app/spam.py - 서버 시작 시 자동 로드
model_path = Path(__file__).resolve().parents[1] / "models" / "spam_pipeline.pkl"
if model_path.exists():
    with open(model_path, "rb") as f:
        _MODEL = pickle.load(f)
        _MODEL_VERSION = "ml-v2"
else:
    _MODEL_VERSION = "keyword-v1" # 키워드 폴백
```

- 서버 시작 시 `models/spam_pipeline.pkl` 존재 여부 확인
- 존재하면 ML 모델 사용 (v2), 없으면 키워드 방식 폴백 (v1)

신규 모델 반영 방법

방법 1: 자동 (GitHub Actions)

```
git push → mlflow-train.yml 실행 → 신규 모델 학습 → models/spam_pipeline.pkl 커밋 → Render 재배포
```

방법 2: 수동 핫 리로드 (재시작 없이)

```
curl -X POST https://gunhee-homework.onrender.com/model/reload  
# 응답: {"model_version": "ml-v2", "status": "reloaded"}
```

자동 반영/수동 반영

- **기본 방식:** `git push` 시 GitHub Actions가 자동으로 모델 재학습 후 커밋, Render 자동 재배포
- **이유:** 코드 변경과 모델 변경이 동일 파이프라인에서 관리되어 일관성 유지
- **핫 리로드**(`/model/reload`)는 배포 없이 즉시 반영이 필요한 긴급 상황에 사용

10. 재학습 또는 모델 개선 과정

어떤 이유로 재학습했는가

총 3번의 학습 실험을 진행하였다.

- 실험 1: 기본 파라미터로 초기 ML 모델 구축 (V1 키워드 방식 대비 성능 비교)
- 실험 2: `max_features=200` , `ncgram=1` , `alpha=2.0` 으로 파라미터 변경하여 경량 모델 테스트
- 실험 3: 실험 1의 최적 파라미터 재확인, Production 승격 대상 선정

데이터/코드/파라미터 중 무엇이 바뀌었는가

구분	실험 1	실험 2	실험 3
max_features	1000	200	1000
ncgram_range	(1,2)	(1,1)	(1,2)
nb_alpha	1.0	2.0	1.0
변경 요소	기준값	**파라미터 축소**	최적값 재확인

재학습 전후 성능 비교

```
실험 실행 결과 (train.py 출력):

V1 (keyword) accuracy=0.8500  f1=0.8235  ← 기준 성능
V2 (ML)      accuracy=1.0000  f1=1.0000  ← 모든 실험에서 항상 확인
```

	V1 키워드	V2 ML (실험1)	V2 ML (실험2)	V2 ML (실험3)
--	--------	-------------	-------------	-------------

Accuracy	85.0%	100.0%	100.0%	100.0%
F1	82.4%	100.0%	100.0%	100.0%
파라미터 수	1개(임계 값)	max_features=1000	max_features=200	max_features=1000

모델 교체 결과

- 실험 3 기준 모델(SpamClassifier v3)을 Production으로 승격
- `models/spam_pipeline.pkl` 최종 버전으로 교체
- 서비스 재시작 후 `/model/info` 응답: `{"model_version": "ml-v2"}`

11. 운영 로그 및 문제 대응

서비스 로그 확인

`app/main.py` 에서 구조화 로그를 기록한다.

로그 포맷: 시간 | LEVEL | 파일:줄 (함수) | 메시지

예시:

```
2026-06-17 15:30:01,123 | INFO | main.py:52 (classify) | CALL /classify | text='free prize' | len=10
```



```
2026-06-17 15:30:01,145 | INFO | main.py:59 (classify) | OK /classify | model=ml-v2 | l
abel=spam score=0.9342
```

예측 요청 로그

모든 `/classify` 요청에 대해:

- 입력 텍스트와 길이
- 현재 모델 버전
- 분류 결과 및 확률값
- 처리 결과 (OK / FAIL)

모델 정보 확인

```
curl http://localhost:10000/model/info
# {"model_version": "ml-v2"}
```

일부러 발생시킨 문제 및 해결

문제 1: 의도적 크래시 테스트

요청 body에 `"text": "crash"` 전송:



```
FAIL /classify | text='crash' | error=RuntimeError: 의도적 장애 추가
```

결과:

- 에러 로그 자동 기록
- GitHub Issue 자동 생성 (`[Prod Error] /classify failed: RuntimeError`)
- API 응답: `{"label": "error", "score": -1.0}`

원인: `app/main.py` 의 `if text == "crash": raise RuntimeError(...)`

해결: 장애를 감지하고 서비스가 계속 운영되는 것을 확인 (GitHub Issue로 팀 알림)

SpamCheck API 분류 결과

모델: ml-v2 (TF-IDF + Naive Bayes)

입력: "Congratulations! You have WON a FREE prize. Click here to claim your CASH reward now!"

 **SPAM**

스팸 확률: 97.8%

입력: "Can we reschedule the meeting to 3pm tomorrow? I'll be a bit late."

 **HAM (정상)**

스팸 확률: 2.3%

입력: "crash" (의도적 장애 유발)

 **에러 → GitHub Issue 자동 생성**

응답: { "label": "error", "score": -1.0 }

12. 롤백 및 이전 모델 관리

이전 모델 보관 여부

MLflow Model Registry에 모든 실험 버전이 영구 보관된다.

버전	Stage	Run ID	파라미터
v1	Archived	12b470dc...	max_features=1000, ngram=2, alpha=1.0
v2	None	6c97caf9...	max_features=200, ngram=1, alpha=2.0
v3	**Production**	25517d26...	max_features=1000, ngram=2, alpha=1.0

이전 모델로 되돌리는 방법

`scripts/rollback.py` 를 사용한다.

```
# 1. 전체 버전 목록 확인
python scripts/rollback.py --list

# 2. v1으로 롤백
python scripts/rollback.py --version 1

# 출력:
# [rollback] v3 archived (was Production)
# [rollback] v1 is now PRODUCTION

# 3. 다시 v3으로 복원
python scripts/rollback.py --version 3
```

모델 버전 관리 화면

mlflow2.17.2ExperimentsModels

GitHubDocs

Registered Models

Create Model

Filter registered models by name or tags

Name	Latest version	Created	Tags
SpamClassifier	Version 3	17 15:52:31	—

Flexible, governed deployments with the new Model Registry UI

Previous experience

Ver 2Production

Ver 1Staging

New Model Registry UI

Ver 2@Champion@LatestApproved

Ver 1@ChallengerIn-Review

With the latest Model Registry UI, you can use **Model Aliases** for flexible references to specific model versions, streamlining deployment in a given environment. Use **Model Tags** to annotate model versions with metadata, like the status of pre-deployment checks.

Learn more

Try it now

New model registry UI

< PreviousNext >25 / page

```
git log --oneline (C:\W...SpamCheck)

4aa787d feat: upgrade to ML model (TF-IDF + Naive Bayes) with full MLOps pipeline
10fad47 fix: add from __future__ import annotations for Python 3.8 compatibility
e58c084 fix: run MLFlow server inside Actions runner to avoid ngrok host header issue
edfc688 Add MLFlow auto train workflow
c2c4e4d Add logging and GitHub issue reporting
13047e4 Dockerize for deployment
4c3c212 Merge pull request #3 from logan0741/fix/issue-3-swagger-docs
6639583 fix: add request schema for classify docs
8e181db Merge pull request #1 from logan0741/feature/ui-title
8c57c34 ui: improve title visibility
cf8799e Merge branch 'feature/ui-soomi'
b6758c9 ui: clarify title with AI keyword
2e58588 ui: translate title
e0ac00c feature: extend spam keywords
787bfa1 resolve: merge UI title changes
ac935e5 ui: clarify title with AI keyword
```

13. 전체 파이프라인 동작 흐름

코드 수정 후 서비스 반영까지

1. 로컬에서 app/ 코드 수정
2. `git commit -m "feat: ..."`
3. `git push origin master`
4. GitHub Actions 자동 실행:
 - └── ci.yml: pytest 9개 테스트 통과 확인
 - └── docker-verify.yml: Docker 빌드 + 컨테이너 헬스체크
5. Render.com: master push 감지 → Docker 재빌드 → 자동 배포
6. 약 2~3분 후 서비스에 변경사항 반영

데이터 변경 후 재학습까지

1. data/dataset.py에 새 데이터 추가
2. `git commit + push`
3. mlflow-train.yml 자동 실행:
 - └── train.py: 새 데이터로 모델 재학습 → MLflow 새 버전 등록
4. models/spam_pipeline.pkl 자동 업데이트 (`git commit`)
5. (선택) scripts/promote.py 실행 → 정확도 기준 Production 승격

모델 변경 후 운영 반영까지

1. 신규 모델 학습 완료 (MLflow Registry에 새 버전 등록)

2. scripts/promote.py 실행 → v4 Production 승격
3. models/spam_pipeline.pkl이 최신 모델로 커밋됨
4. Render 재배포 완료 후:
curl -X POST /model/reload # 서버 재시작 없이 즉시 반영
5. /model/info 확인 → "ml-v2" 응답으로 새 모델 로드 확인

14. 문제 해결 경험

문제 1: Python 3.8 타입 힌트 호환성 오류

발생 문제: app/spam.py 에서 tuple[str, int] 타입 힌트 사용 시 Python 3.8에서 오류 발생

```
# 오류 발생 코드 (Python 3.9+ 전용 문법)
def check_spam(text: str) -> tuple[str, int]:
```

원인 분석: Python 3.9부터 내장 타입을 직접 제네릭으로 사용 가능. 3.8에서는 typing.Tuple 사용 필요

해결 방법: from __future__ import annotations 추가

```
from __future__ import annotations # 추가
```

```
def check_spam(text: str) -> tuple[str, int]:  
    ...
```

이 import는 모든 타입 힌트를 런타임에 평가하지 않고 문자열로 처리하여 3.9+ 문법을 3.8에서 사용 가능하게 함

문제 2: MLflow DB 버전 충돌

발생 문제: 기존 `mlruns/mlflow.db` 가 있는 상태에서 새 버전의 MLflow로 SQLite 트래킹을 시도하자 마이그레이션 오류 발생

```
alembic.util.exc.CommandError: Can't locate revision identified by '7d34483879f0'
```

원인 분석: 이전 MLflow 버전으로 생성된 SQLite DB의 마이그레이션 히스토리가 현재 설치된 MLflow 버전과 불일치

해결 방법: `MLFLOW_TRACKING_URI` 를 SQLite URI 대신 빈 값으로 설정하여 파일 기반 추적 사용

```
# train.py 수정  
tracking_uri = os.getenv("MLFLOW_TRACKING_URI", "")
```



```
if tracking_uri:
    mlflow.set_tracking_uri(tracking_uri)
# 기본값: ./mlruns/ 파일 기반 추적 (DB 불필요)
```

문제 3: GitHub Actions에서 MLflow 서버 연결 실패

발생 문제: GitHub Actions runner에서 외부 MLflow 서버(ngrok 터널)에 연결 시 Host 헤더 검증 실패

```
MLflowException: INVALID_PARAMETER_VALUE: Invalid host header
```

원인 분석: MLflow 2.x의 보안 정책으로 허용되지 않은 Host 헤더 거부

해결 방법: GitHub Actions runner 내부에 MLflow 서버를 직접 실행하고 localhost로 연결

```
- name: Start MLFlow tracking server
  run: |
    mlflow server --host 0.0.0.0 --port 5000 --backend-store-uri sqlite:///mlflow.db &
    sleep 5
- name: Train model
  env:
    MLFLOW_TRACKING_URI: http://localhost:5000
  run: python train.py
```

15. 느낀 점 및 개선 방향

MLOps 관점에서 배운 점

이번 프로젝트를 통해 단순히 ML 모델을 학습하는 것과, 실제 서비스에 ML을 통합하여 운영하는 것이 얼마나 다른지 체감하였다.

1. **재현성의 중요성**: Docker로 환경을 고정하지 않으면 로컬에서는 되는데 서버에서 안 되는 문제가 반복된다. `FROM python:3.10-slim`으로 이미지를 고정하는 것만으로 많은 문제가 해결되었다.

2. **MLflow는 모델 개발의 일기장**: 파라미터, 메트릭, 아티팩트를 자동으로 기록하면 "왜 이 모델이 더 좋은가"를 나중에 설명할 수 있다. 기록하지 않으면 실험 간 비교가 불가능하다.

3. **CI/CD는 신뢰의 근거**: `git push` 마다 테스트가 자동 실행되면 "코드가 깨졌는지" 걱정 없이 수정할 수 있다. 없을 때와 있을 때의 심리적 차이가 크다.

4. **로그와 알림의 가치**: 서버에서 오류가 나도 GitHub Issue로 즉시 알림이 오면 빠른 대응이 가능하다. 운영에서 로그는 선택이 아닌 필수이다.

5. **모델 롤백 준비**: 새 모델을 배포했다가 문제가 생기면 빠르게 이전 버전으로 돌아가야 한다. MLflow Registry로 버전을 관리하고 `rollback.py`를 미리 준비해두면 롤백이 스크립

트 하나로 해결된다.

개선하고 싶은 부분

1. **실제 데이터셋 도입**: 현재 100개의 수동 데이터셋 대신 SMS Spam Collection(5,574개) 같은 공개 데이터셋을 사용하면 더 견고한 모델을 구축할 수 있다.
2. **MLflow 서버 공개 배포**: 현재 MLflow는 로컬 파일 기반으로 추적 중이다. Render.com에 MLflow 서버를 별도 배포하면 GitHub Actions에서 원격 서버로 실험 결과를 영구 보관할 수 있다.
3. **모델 드리프트 감지**: 서비스 운영 중 스팸 패턴이 변하면 모델 성능이 저하된다. 예측 로그를 분석하여 정확도가 일정 수준 이하로 떨어지면 자동 재학습을 트리거하는 구조가 필요하다.
4. **A/B 테스트**: V1(키워드)과 V2(ML) 모델을 실제 트래픽으로 비교하면 더 객관적인 성능 비교가 가능하다.
5. **한국어 스팸 지원**: 현재 영어 데이터 기반이다. 한국어 스팸(보이스피싱, 대출광고 등)을 처리하려면 한국어 토큰나이저(KoNLPy)와 한국어 학습 데이터가 필요하다.

수업 피드백

- 실습 중심의 수업 구성 덕분에 Git, Docker, GitHub Actions, MLflow가 실제로 어떻게 연결되는지 이해할 수 있었다.

- 각 실습이 독립적으로 보였지만 기말 프로젝트에서 모두 하나의 파이프라인으로 합쳐지는 경험이 매우 인상적이었다.
 - MLflow Model Registry와 롤백 개념을 더 일찍 배웠다면 중간 실습부터 적용해볼 수 있었을 것 같다.
-

참고 자료

- 수업 자료: [10_2_MLOps 개요.pdf](#) , [11_1_MLSpamchecker와 MLFlow 개요.pdf](#) , [11_2_MLFlow 로컬 실습.pdf](#) , [12_1_MLFlow 서버 실습.pdf](#) , [12_2_GitHub Actions 기반 자동 훈련.pdf](#)
- [MLflow 공식 문서](#)
- [scikit-learn Pipeline 문서](#)
- [FastAPI 공식 문서](#)
- [Render.com Docker 배포 가이드](#)
- GitHub 저장소: https://github.com/logan0741/gunhee_homework